

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

**Sistema LLM para análisis financiero histórico
mediante generación automática de código**

MÁSTER DE FORMACIÓN PERMANENTE EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este trabajo estudia el diseño de un sistema multiagente basado en modelos de lenguaje para realizar análisis financiero histórico a partir de consultas formuladas en lenguaje natural. El foco del proyecto no está en predecir el mercado, sino en organizar un flujo trazable en el que la petición se transforme en una estructura de datos interpretable, se validen los datos necesarios, se genere código ejecutable, se controle su ejecución y se redacte una respuesta final apoyada en evidencias.

La memoria queda preparada para documentar ese recorrido por etapas. En particular, se refuerzan la metodología, la descripción del flujo y el marco de evaluación, de manera que el trabajo final pueda explicar no solo si el sistema produce respuestas, sino también cómo llega a ellas, dónde falla y qué garantías ofrece cada bloque del proceso.

En esta versión, varias secciones se han dejado como plantilla guiada para completar conforme se implemente y evalúe el flujo definitivo. El objetivo es que la memoria final mantenga coherencia entre diseño, ejecución, resultados y limitaciones, y que cada afirmación importante pueda apoyarse en artefactos observables del sistema.

Palabras clave: análisis financiero histórico, modelos de lenguaje, generación de código, agentes LLM, ejecución controlada, trazabilidad.

Abstract

This work studies the design of a multi-agent language-model-based system for historical financial analysis from natural-language queries. Rather than predicting market behaviour, the project focuses on defining a traceable workflow in which a request is transformed into structured data, relevant market data are validated, executable code is produced, execution is controlled, and the final answer is grounded in observable evidence.

The dissertation has been prepared so that methodology, workflow description and evaluation can be developed rigorously as the final system is implemented. Several sections are therefore kept as guided templates, allowing the final document to explain not only whether the system works, but also how each stage contributes to robustness, traceability and controlled failure handling.

At this stage, the document provides a coherent structure for reporting the final project, with special emphasis on methodological justification, stage-by-stage analysis and evidence-based evaluation.

Keywords:

historical financial analysis, language models, code generation, LLM agents, traceability.

Índice general

Resumen	I
Abstract	II
Abreviaturas y acronimos	VIII
1. Introduccion	1
1.1. Contexto y definicion del problema	1
1.2. Motivacion	2
1.3. Objetivos	2
1.4. Alcance	3
1.5. Estructura de la memoria	3
2. Estado de la cuestion	4
2.1. Enfoque de la revision	4
2.2. Modelos de lenguaje y agentes financieros	4
2.3. Agentes, herramientas y orquestacion	5
2.4. Generacion y ejecucion controlada de codigo	5
2.5. Fiabilidad y evaluacion de sistemas generativos	6
2.6. Comparativa y posicionamiento de la propuesta	7
3. Material de trabajo y planteamiento experimental	8
3.1. Funcion de este capitulo	8
3.2. Origen y naturaleza del material de trabajo	8
3.3. Preguntas utiles para orientar este capitulo	8
3.4. Inventario del material	9
3.5. Casos y escenario experimental de partida	9
3.6. Criterio de redaccion recomendado	9
4. Metodologia	10
4.1. Funcion de este capitulo	10
4.2. Enfoque metodologico	10

4.3.	Preguntas guia para escribir bien la metodologia	10
4.3.1.	Sobre el objetivo de evaluacion	10
4.3.2.	Sobre la unidad de analisis	11
4.3.3.	Sobre el diseno experimental	11
4.3.4.	Sobre la evaluacion	11
4.3.5.	Sobre la validez del estudio	11
4.4.	Estructura recomendada para la metodologia final	12
4.4.1.	Pregunta de investigacion y objetivos evaluables	12
4.4.2.	Unidad de analisis	12
4.4.3.	Diseno experimental	12
4.4.4.	Protocolo de ejecucion	12
4.4.5.	Metricas de evaluacion	12
4.4.6.	Revision humana	12
4.4.7.	Amenazas a la validez	13
4.5.	Criterio de calidad para este capitulo	13
5.	Flujo multiagente del sistema	14
5.1.	Funcion de este capitulo	14
5.2.	Preguntas que debe responder el capitulo	14
5.3.	Vision general del flujo	15
5.4.	Entrada y estado compartido	16
5.4.1.	Entrada del sistema	16
5.4.2.	Estado compartido	17
5.5.	Flujo principal del sistema	18
5.5.1.	Parte de datos	18
5.5.2.	Analista y generador de codigo	19
5.5.3.	Validacion del codigo	20
5.5.4.	Validacion de ejecucion	21
5.5.5.	Interpretacion y respuesta final	21
5.6.	Subagentes de correccion y recuperacion	21
5.7.	Diseno de prompts y contratos de los agentes	21
5.8.	Trazabilidad y artefactos generados	22
5.9.	Decisiones de diseno	23
6.	Resultados y evaluacion	24
6.1.	Funcion de este capitulo	24
6.2.	Enfoque recomendado	24
6.3.	Preguntas que convendra responder mas adelante	24
6.4.	Plantilla de estructura futura	25
6.4.1.	Resultado global	25

6.4.2.	Lectura por grupos de casos	25
6.4.3.	Análisis por etapas	25
6.4.4.	Casos representativos	25
6.4.5.	Discusión crítica	25
7.	Limitaciones, aspectos éticos y reproducibilidad	26
7.1.	Función de este capítulo	26
7.2.	Limitaciones del estudio	26
7.3.	Aspectos éticos y uso responsable	26
7.4.	Reproducibilidad	26
7.5.	Preguntas de control	27
8.	Conclusiones y trabajo futuro	28
8.1.	Función de este capítulo	28
8.2.	Guía para las conclusiones	28
8.3.	Guía para el trabajo futuro	28
A.	Anexos	29
A.1.	Ejecución del sistema	29
A.1.1.	Preparación mínima	29
A.1.2.	Ejecución con ejemplos integrados	29
A.1.3.	Ejecución con una entrada JSON propia	30
A.1.4.	Salida y artefactos generados	30
A.2.	Configuración LLM y API	31
A.2.1.	Configuración recomendada	31
A.2.2.	Variables equivalentes admitidas	31
A.2.3.	Parámetros de generación	32
A.2.4.	Comprobaciones prácticas	32
A.3.	Notas para reproducibilidad	32

Índice de tablas

2.1. Comparacion entre los trabajos relacionados y la propuesta desarrollada.	7
3.1. Inventario inicial del material de trabajo.	9
4.1. Metricas previstas para la evaluacion del sistema.	12
5.1. Guia para documentar los prompts y contratos de los agentes.	22

Índice de figuras

5.1. Flujo multiagente de referencia para el desarrollo del sistema de analisis financiero.	15
--	----

Abreviaturas y acronimos

API Application Programming Interface.

CSV Comma-Separated Values.

ETF Exchange-Traded Fund.

LLM Large Language Model.

OHLCV Open, High, Low, Close, Adjusted Close y Volume.

RAG Retrieval-Augmented Generation.

UTC Coordinated Universal Time.

Capítulo 1

Introduccion

1.1. Contexto y definicion del problema

Los modelos de lenguaje de gran escala han ampliado de forma notable las posibilidades de interaccion con los sistemas informaticos. Ademas de producir texto, pueden interpretar instrucciones, proponer procedimientos y generar codigo ejecutable. Buena parte de este avance se apoya en la arquitectura Transformer y en el mecanismo de atencion, que constituyen una de las bases de los modelos de lenguaje actuales.

La capacidad de generar codigo resulta especialmente interesante en tareas de analisis de datos. Una consulta expresada en lenguaje natural puede transformarse en operaciones de carga, filtrado, calculo y presentacion de resultados. Sin embargo, esta flexibilidad introduce un problema de confianza: que el codigo sea plausible no significa que sea correcto, y que una respuesta este bien redactada no garantiza que respete los datos de partida.

Este problema cobra especial importancia en el ambito financiero. Incluso cuando el analisis se limita a informacion historica, una cifra incorrecta o una interpretacion poco prudente puede conducir a conclusiones equivocadas. Por ello, no basta con pedir al modelo que responda directamente a una consulta: tambien es necesario conocer que datos ha utilizado, que metricas ha calculado, que codigo ha ejecutado y que limitaciones presenta el resultado.

Este trabajo aborda el problema desde la integracion de sistemas, no desde el entrenamiento de un nuevo modelo. Para ello, se desarrolla un prototipo que recibe una consulta financiera y un conjunto de datos historicos almacenados en CSV, organiza el analisis mediante agentes LLM y mantiene los calculos dentro de un flujo programatico. El sistema separa la planificacion, la generacion de codigo y la interpretacion final, valida el codigo antes de ejecutarlo y conserva los principales artefactos de cada ejecucion.

1.2. Motivacion

El proyecto nace del interes por combinar tres ambitos que a menudo se estudian por separado: los modelos de lenguaje, la programacion para el analisis de datos y la interpretacion financiera. Los LLM permiten construir con rapidez interfaces capaces de entender peticiones diversas, pero esa misma facilidad puede ocultar errores cuando todo el proceso queda reducido a una unica respuesta generativa.

La motivacion principal es comprobar hasta que punto puede aprovecharse la flexibilidad del modelo sin renunciar a mecanismos basicos de control. En el sistema propuesto, el LLM decide como abordar la consulta y genera el codigo necesario, pero el resultado financiero procede de la ejecucion de ese codigo sobre datos locales. Esta separacion permite revisar tanto el planteamiento seguido como los calculos obtenidos.

Otro criterio importante es que los fallos sean visibles. Si el prototipo no puede generar codigo valido, la ejecucion no termina correctamente o la salida supera las capacidades del modelo, el sistema registra el problema y devuelve un error controlado. Este comportamiento resulta preferible a completar una respuesta con cifras no verificadas. La evaluacion mediante los casos de estudio definidos para el proyecto permite observar estos fallos, especialmente en los escenarios de mayor dificultad, e identificar en que parte del proceso se producen.

El prototipo se construyo de forma incremental, desde consultas sencillas hasta un flujo con tres agentes, validacion, reparacion y conservacion de evidencias.

1.3. Objetivos

El objetivo general del trabajo es disenar, implementar y evaluar un sistema basado en un modelo de lenguaje capaz de transformar consultas sobre datos financieros historicos en calculos ejecutables y respuestas trazables.

Los objetivos especificos son:

- Disenar una arquitectura modular que separe la planificacion, la generacion de codigo, su validacion, la ejecucion de los calculos y la elaboracion de la respuesta final.
- Desarrollar un flujo capaz de interpretar consultas con diferentes requisitos analiticos y determinar los datos, metricas y operaciones necesarios.
- Generar y ejecutar codigo Python adaptado a cada consulta, aplicando controles que limiten las operaciones permitidas.
- Garantizar la trazabilidad del proceso mediante la conservacion del plan, el codigo generado, los resultados estructurados y los registros de error.

- Generar respuestas financieras basadas exclusivamente en los calculos realizados, sin incorporar predicciones ni recomendaciones de inversion.
- Evaluar la capacidad del modelo de lenguaje para comprender y adaptarse a las tareas planteadas, generar codigo acorde con sus requisitos y redactar respuestas claras, precisas y fundamentadas.

1.4. Alcance

El sistema se limita al analisis historico y descriptivo de activos financieros a partir de datos descargados previamente y almacenados en ficheros CSV. Los casos utilizados incluyen acciones, indices, fondos cotizados, divisas, una materia prima y un criptoactivo, con diferentes periodos e intervalos temporales.

El prototipo no obtiene precios en tiempo real durante la ejecucion, no incorpora noticias ni informacion macroeconomica y tampoco realiza analisis fundamental. No pretende predecir movimientos futuros ni recomendar la compra o venta de activos. Estas restricciones responden al proposito del trabajo: evaluar una arquitectura LLM trazable, no construir un asesor financiero.

1.5. Estructura de la memoria

La memoria se organiza en torno a los tres bloques que mas peso tendran en el desarrollo del TFM: la metodologia, el flujo multiagente y la evaluacion del sistema. El Capitulo 2 revisa los trabajos previos y situa la propuesta dentro del estado de la cuestion. El Capitulo 3 resume el material de trabajo y el planteamiento experimental de partida.

El Capitulo 4 define la metodologia con la que se estudiara el sistema: unidad de analisis, protocolo de ejecucion, criterios de evaluacion y amenazas a la validez. El Capitulo 5 se reserva para el flujo multiagente del sistema y para la explicacion detallada de sus etapas, contratos, validaciones y rutas de recuperacion. El Capitulo 6 recogerá los resultados y la evaluacion final del proyecto.

Los ultimos capitulos se plantean de forma mas compacta. El Capitulo 7 reunira las limitaciones, los aspectos eticos y las condiciones de reproducibilidad, mientras que el Capitulo 8 cerrara la memoria con las conclusiones y el trabajo futuro. Por ultimo, los anexos conservaran solo la informacion operativa de apoyo que siga siendo util para ejecutar el sistema, documentar su configuracion LLM y describir los artefactos tecnicos que deja cada corrida.

Capítulo 2

Estado de la cuestion

2.1. Enfoque de la revision

Este capitulo situa el trabajo dentro del uso de modelos de lenguaje para el analisis financiero y la automatizacion de tareas de datos. No se pretende revisar todo el campo de la inteligencia artificial aplicada a las finanzas, sino centrarse en las lineas que sustentan el sistema desarrollado: los modelos especializados en este dominio, las arquitecturas basadas en agentes, el uso de herramientas externas, la generacion de codigo y la evaluacion de sistemas generativos.

El punto de partida es un modelo de pesos abiertos ya existente, identificado en este trabajo como `openai/gpt-oss-20b` e integrado en un flujo de trabajo controlado. Ante una consulta en lenguaje natural, el modelo interviene en la planificacion del analisis, la generacion de codigo Python y la redaccion de una respuesta basada en los resultados obtenidos al analizar datos historicos locales. Quedan fuera del alcance el entrenamiento de un modelo propio, la prediccion de precios y la toma autonoma de decisiones de inversion.

2.2. Modelos de lenguaje y agentes financieros

La aplicacion del procesamiento del lenguaje natural al ambito financiero es anterior a los modelos generativos actuales. FinBERT adapta la arquitectura BERT al lenguaje financiero y muestra la utilidad de los modelos especializados en tareas como el analisis de sentimiento. Posteriormente, BloombergGPT y FinGPT ampliaron esta linea mediante modelos entrenados o adaptados con noticias, documentacion corporativa y otros recursos propios del sector.

Estos trabajos muestran que el lenguaje financiero contiene vocabulario y relaciones que requieren una interpretacion cuidadosa. En este trabajo se adopta una perspectiva distinta: se utiliza `openai/gpt-oss-20b`, un modelo de pesos abiertos con cerca

de 21.000 millones de parametros totales que, gracias a su arquitectura de mezcla de expertos, activa aproximadamente 3.600 millones por cada token procesado. La denominacion *20b* indica su escala aproximada, pero no constituye por si sola una medida de la calidad de las respuestas. El interes se centra en estudiar su integracion en un sistema verificable de analisis historico.

La evolucion de los LLM financieros tambien ha favorecido la aparicion de sistemas basados en agentes. FinRobot propone una arquitectura orientada a la investigacion y valoracion de empresas, mientras que TradingAgents distribuye tareas de analisis, debate y toma de decisiones de trading entre varios agentes. En ambos casos, el modelo deja de actuar unicamente como generador de texto y pasa a intervenir en un proceso compuesto por distintas responsabilidades.

2.3. Agentes, herramientas y orquestacion

Los sistemas basados en agentes permiten que un modelo vaya mas alla de la generacion de texto: puede utilizar herramientas y trabajar con los resultados de acciones externas. ReAct propone alternar razonamiento y actuacion para que el modelo observe las consecuencias de sus decisiones. Toolformer estudia como puede aprender a recurrir a herramientas cuando las necesita. Las revisiones sobre agentes basados en LLM describen esta evolucion como el paso de sistemas puramente conversacionales a arquitecturas capaces de planificar, actuar y utilizar observaciones del entorno.

Para coordinar estas tareas es necesario conservar la informacion producida durante la ejecucion. En la practica, esto exige mantener un estado compartido, definir etapas con responsabilidades separadas y establecer transiciones controladas entre validacion, generacion, ejecucion e interpretacion. Esta estructura resulta especialmente util cuando se combinan componentes generativos con comprobaciones programaticas.

El sistema adopta este patron mediante un recorrido definido de antemano. El modelo no selecciona libremente las herramientas ni altera el orden del proceso, y el estado compartido conserva la informacion de cada ejecucion sin mantener una memoria conversacional permanente. Los detalles de esta arquitectura se presentan mas adelante en los capitulos dedicados a la metodologia y al flujo multiagente del sistema.

2.4. Generacion y ejecucion controlada de codigo

La generacion automatica de codigo es una de las capacidades que mas ha impulsado el uso de los modelos de lenguaje en tareas tecnicas. El trabajo sobre Codex mostro que un modelo entrenado con grandes cantidades de codigo puede sintetizar programas a partir de descripciones en lenguaje natural y evaluarse con pruebas funcionales como HumanEval. En ciencia de datos, DS-1000 amplio esta linea con un banco de pruebas

basado en problemas mas cercanos a escenarios reales y apoyado en librerias habituales de analisis.

Estas investigaciones respaldan el uso de un LLM para traducir una consulta financiera en operaciones de carga, filtrado y calculo. Tambien ponen de manifiesto una limitacion fundamental: que un programa parezca correcto no implica que sea ejecutable, seguro o fiel a la peticion. Por ello, el codigo no se considera una respuesta final, sino un artefacto intermedio que debe superar varios controles.

Ejecutar codigo generado por un modelo introduce riesgos especificos. Los estudios sobre asistentes de programacion han mostrado que sus sugerencias pueden contener vulnerabilidades, incluso cuando parecen funcionales. Por este motivo, el codigo debe validarse antes de su ejecucion y limitarse a un conjunto restringido de operaciones.

La propuesta trata el codigo como un artefacto intermedio que debe respetar un contrato estructurado y superar controles antes de ejecutarse. Esta combinacion permite separar el calculo de su comunicacion, aunque no elimina todos los riesgos asociados a la ejecucion de codigo generado.

2.5. Fiabilidad y evaluacion de sistemas generativos

Una limitacion conocida de los modelos generativos es la produccion de respuestas plausibles que no se corresponden con la informacion disponible, fenomeno denominado habitualmente alucinacion. La generacion aumentada con recuperacion busca reducir este problema apoyando la respuesta en fuentes obtenidas de forma explicita. Aunque el sistema no incorpora recuperacion documental, sigue un principio de anclaje similar: la explicacion final debe basarse en datos locales, codigo ejecutado y resultados registrados.

Este anclaje es especialmente importante en el analisis financiero. Cuando una respuesta presenta datos de rentabilidad, volatilidad o drawdown, debe ser posible revisar tanto la informacion de partida como el procedimiento seguido. Tambien conviene separar con claridad la descripcion del comportamiento historico de cualquier prediccion o recomendacion. Conservar la consulta, el plan, el codigo, los resultados, los errores y la respuesta final no elimina la posibilidad de fallo, pero ayuda a localizarlo y valorar sus consecuencias.

La evaluacion de un sistema de estas caracteristicas no puede reducirse a una unica cifra. Los marcos de evaluacion amplios y transparentes defienden una lectura compuesta por distintos escenarios y metricas, para evitar que un resultado agregado oculte capacidades y limitaciones relevantes. En el caso del texto generado, tambien se ha senalado que las metricas automaticas tradicionales no siempre reflejan la calidad percibida durante una evaluacion humana.

Siguiendo estas ideas, la evaluacion combina indicadores tecnicos, una valoracion

cualitativa y revision humana. Esta aproximacion permite distinguir entre los fallos del flujo y las respuestas que, aun completandose, presentan problemas de pertinencia, fidelidad o claridad.

2.6. Comparativa y posicionamiento de la propuesta

La Tabla 2.1 resume las lineas revisadas y muestra como se relacionan con el sistema desarrollado. El objetivo concreto es integrar un modelo de lenguaje con generacion controlada de codigo y analisis reproducible de datos historicos.

Tabla 2.1: Comparacion entre los trabajos relacionados y la propuesta desarrollada.

Trabajo o linea	Aportacion principal	Relacion con este trabajo
FinBERT, BloombergGPT y FinGPT	Modelos entrenados o adaptados al lenguaje financiero.	Se centran en el modelo; la propuesta estudia la integracion controlada de openai/gpt-oss-20b en un flujo verificable.
FinRobot y TradingAgents	Agentes para investigacion, valoracion o trading.	Inspiran la separacion de responsabilidades, pero el sistema no toma decisiones de inversion ni actua sobre el mercado.
ReAct, Toolformer y arquitecturas de orquestacion	Uso de herramientas externas y coordinacion de pasos mediante observaciones y transiciones controladas.	Respaldan el uso de herramientas y un estado comun; aqui el recorrido esta fijado para favorecer el control.
Codex, DS-1000 y seguridad de codigo	Sintesis, evaluacion y riesgos del codigo producido por modelos.	Justifican la generacion de codigo bajo contrato y su validacion antes de ejecutarlo.
Alucinaciones y RAG	Estudio de respuestas no fieles y anclaje en fuentes externas.	La respuesta se apoya en los CSV locales y en los resultados obtenidos, aunque no se utiliza recuperacion documental.
HELM y G-Eval	Evaluacion multidimensional y calidad del texto generado.	Fundamentan la combinacion de indicadores tecnicos, dimensiones cualitativas y revision humana.

En conjunto, el trabajo ocupa una posicion intermedia entre un asistente puramente conversacional y un agente financiero autonomo. Un unico modelo planifica el analisis, genera el codigo y explica los resultados, mientras que las capas deterministas se encargan de la validacion, la ejecucion y el registro de artefactos. Esta separacion permite estudiar el valor practico del modelo sin atribuirle capacidades que no se han implementado ni evaluado.

Capítulo 3

Material de trabajo y planteamiento experimental

3.1. Funcion de este capitulo

Este capitulo debe situar al lector antes de entrar en la metodologia formal y antes de describir el flujo del sistema. Su papel es explicar con que material se va a trabajar, que tipo de consultas se quieren resolver y por que ese conjunto resulta adecuado para tensionar el nuevo TFM.

3.2. Origen y naturaleza del material de trabajo

Completar aqui el origen de los datos, el tipo de instrumentos incluidos, la razon de su seleccion y la forma en que se obtendran o congelaran para el proyecto. Conviene mantener esta seccion breve y muy clara.

3.3. Preguntas utiles para orientar este capitulo

1. ¿Que activos o instrumentos quieres incluir en el nuevo TFM y por que esos y no otros?
2. ¿Quieres priorizar variedad de clases de activo, realismo de las consultas o facilidad de reproducir el experimento?
3. ¿Que papel juegan los datos historicos descargados frente a la posible descarga automatica durante el flujo?
4. ¿Las consultas van a nacer de ejemplos cerrados, de una bateria definida o de escenarios abiertos controlados?

5. ¿Que hace que el conjunto de casos sea suficientemente representativo para defender el trabajo?

3.4. Inventario del material

Elemento	Tipo	Estado	Papel dentro del TFM
<i>Completar</i>	<i>Datos / casos / artefactos</i>	<i>Disponible / por construir</i>	<i>Completar</i>
<i>Completar</i>	<i>Datos / casos / artefactos</i>	<i>Disponible / por construir</i>	<i>Completar</i>
<i>Completar</i>	<i>Datos / casos / artefactos</i>	<i>Disponible / por construir</i>	<i>Completar</i>

Tabla 3.1: Inventario inicial del material de trabajo.

3.5. Casos y escenario experimental de partida

Completar aqui el tipo de consultas que quieres estudiar: consultas directas, comparativas, tecnicas, con restricciones de formato, con necesidad de descarga, con validaciones o con reparacion. La idea es anticipar la forma de la evaluacion sin entrar todavia en las metricas formales.

3.6. Criterio de redaccion recomendado

Cuando este capitulo quede cerrado, deberia responder con claridad a tres preguntas:

1. ¿Con que material concreto se construye el proyecto?
2. ¿Que tipo de casos o consultas se espera resolver?
3. ¿Por que ese punto de partida es suficiente para justificar el estudio?

Capítulo 4

Metodologia

4.1. Funcion de este capitulo

Este capitulo debe explicar como se va a estudiar el sistema y con que criterio se va a juzgar si el nuevo TFM funciona bien o no. No debe confundirse con la descripcion del flujo: aqui lo importante no es como esta construido el sistema, sino como se va a analizar de forma defendible.

4.2. Enfoque metodologico

Completar aqui la idea central del enfoque metodologico. Por ejemplo: desarrollo iterativo de un sistema multiagente, evaluacion empirica sobre casos de analisis financiero historico, combinacion de revision tecnica y lectura cualitativa de resultados, etc.

4.3. Preguntas guia para escribir bien la metodologia

4.3.1. Sobre el objetivo de evaluacion

1. ¿Que quieres demostrar exactamente con este TFM?
2. ¿Quieres demostrar viabilidad del flujo, robustez de la arquitectura, calidad de las respuestas o una combinacion de ellas?
3. ¿Que seria un buen resultado, pero no necesariamente una demostracion completa?
4. ¿Que afirmaciones no deberias hacer aunque el prototipo funcione bien?

4.3.2. Sobre la unidad de analisis

1. ¿Que vas a considerar como unidad de analisis principal: una consulta, una ejecucion completa, una etapa del flujo o un caso compuesto?
2. ¿Te interesa medir solo el resultado final o tambien el comportamiento intermedio de cada agente?
3. ¿Quieres analizar los fallos por etapa o solo por caso completo?

4.3.3. Sobre el diseno experimental

1. ¿Como vas a construir o seleccionar la bateria de consultas del nuevo TFM?
2. ¿Como vas a graduar la dificultad de los casos?
3. ¿Quieres hablar de complejidad de forma cualitativa sin imponer una etiqueta fija al sistema?
4. ¿Que tipos de consulta deben estar presentes para que el experimento sea creible?
5. ¿Que parte del flujo quieres tensionar de forma deliberada?

4.3.4. Sobre la evaluacion

1. ¿Como distinguiras entre exito tecnico del flujo y calidad real del analisis?
2. ¿Que metricas tecnicas quieres observar: completitud, tiempo, errores por etapa, reparaciones, trazas conservadas?
3. ¿Que dimensiones cualitativas quieres valorar: pertinencia de metricas, fidelidad de la interpretacion, prudencia, claridad?
4. ¿Habra revision humana? Si la hay, ¿con que rubrica o con que criterios?

4.3.5. Sobre la validez del estudio

1. ¿Que amenazas a la validez interna puede tener el experimento?
2. ¿Que limita la generalizacion del trabajo?
3. ¿Hasta que punto dependes del modelo concreto, de los prompts o del proveedor de inferencia?
4. ¿Como evitaras confundir una respuesta convincente con un resultado correcto?

4.4. Estructura recomendada para la metodologia final

4.4.1. Pregunta de investigacion y objetivos evaluables

Completar aqui la pregunta principal de investigacion y los objetivos que realmente puedan verificarse con el experimento.

4.4.2. Unidad de analisis

Completar aqui que se evalua exactamente en cada caso y por que esa unidad de analisis es adecuada para el TFM.

4.4.3. Diseno experimental

Completar aqui el numero de casos, su clasificacion, su dificultad y el razonamiento que justifica ese diseno. Si se habla de consultas mas simples o mas avanzadas, conviene que esa clasificacion se use solo como herramienta de evaluacion y no como una etiqueta explicita que el sistema reciba en la entrada.

4.4.4. Protocolo de ejecucion

Completar aqui como se lanza cada caso, que artefactos se conservan, que pasos del flujo deben completarse y como se registran errores, reparaciones y tiempos.

4.4.5. Metricas de evaluacion

Metrica	Tipo	Utilidad dentro de la evaluacion
Completar	Tecnica / cualitativa	Completar
Completar	Tecnica / cualitativa	Completar
Completar	Tecnica / cualitativa	Completar

Tabla 4.1: Metricas previstas para la evaluacion del sistema.

4.4.6. Revision humana

Completar aqui si habra rubricado manual, lectura comparada de respuestas, comprobacion de metricas o supervision por etapas.

4.4.7. Amenazas a la validez

Completar aqui sesgos de seleccion de casos, dependencia de configuracion, sensibilidad a prompts y cualquier otra limitacion metodologica importante.

4.5. Criterio de calidad para este capitulo

La metodologia final deberia permitir responder sin ambigüedad a cuatro cuestiones:

1. ¿Que se va a evaluar exactamente?
2. ¿Como se va a ejecutar esa evaluacion?
3. ¿Con que criterios se juzgara el sistema?
4. ¿Que limites tendra la interpretacion de los resultados?

Capítulo 5

Flujo multiagente del sistema

5.1. Funcion de este capitulo

Este capitulo es uno de los nucleos del nuevo TFM. Aqui no basta con decir que el sistema usa varios agentes: hace falta explicar con detalle como circula la informacion, que hace cada bloque, que controles existen y por que esa organizacion mejora la trazabilidad y la explicabilidad del analisis.

La idea central que conviene sostener en todo el capitulo es la siguiente: el valor del sistema no esta solo en producir una respuesta final, sino en hacerlo mediante un flujo auditable, con artefactos observables y con decisiones tecnicas que permitan entender que ha ocurrido en cada etapa.

5.2. Preguntas que debe responder el capitulo

1. ¿Que recibe exactamente el sistema y como se transforma esa entrada a lo largo del flujo?
2. ¿Que hace cada bloque principal y por que ese bloque es necesario?
3. ¿Que diferencia hay entre el flujo principal y los mecanismos de correccion?
4. ¿Que papel tienen los prompts como contrato operativo de los agentes?
5. ¿Que evidencias deja el sistema para poder reconstruir una ejecucion completa?
6. ¿Por que este flujo es mas explicable y controlable que una respuesta directa del LLM?
7. ¿Que decisiones de diseno ayudan a justificar luego la metodologia y la evaluacion?

5.3. Vision general del flujo

La Figura 5.1 se toma como referencia de alto nivel para el desarrollo del nuevo TFM. En ella se representa el recorrido completo desde la solicitud inicial del usuario hasta la respuesta final, incluyendo los agentes principales, los subagentes de correccion y los puntos de validacion mas relevantes.

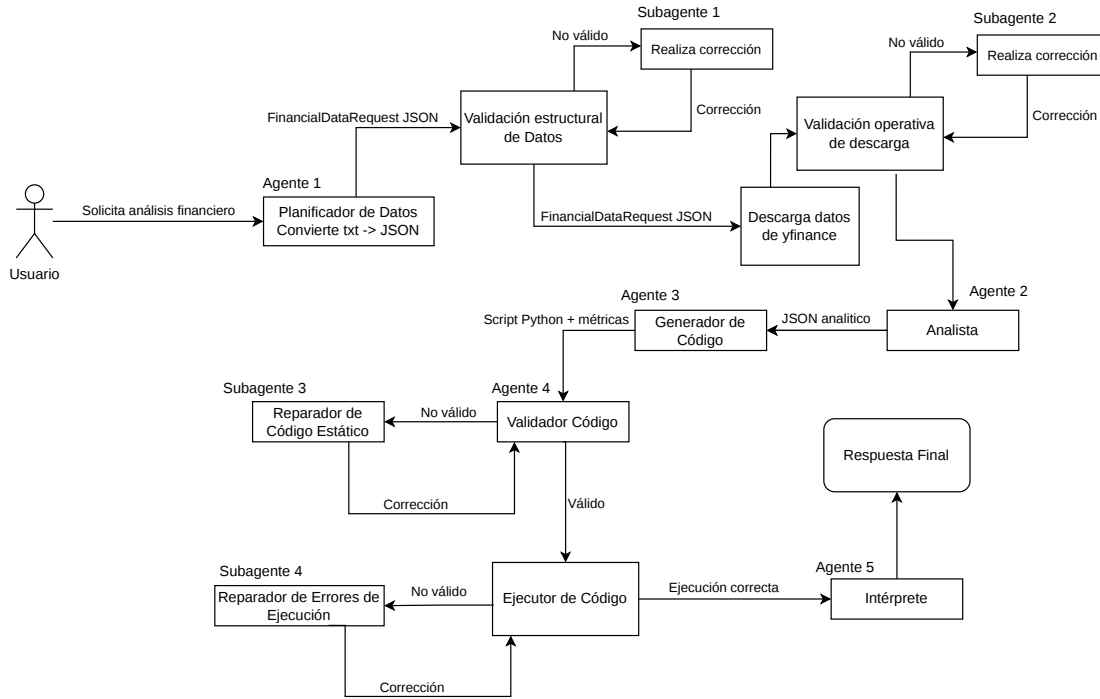


Figura 5.1: Flujo multiagente de referencia para el desarrollo del sistema de analisis financiero.

El flujo parte de una consulta escrita en lenguaje natural y la transforma de manera progresiva en una respuesta final sustentada por datos descargados, decisiones intermedias trazables y una ejecucion controlada. La primera fase se concentra en resolver el problema de datos: el Agente 1 convierte la peticion del usuario en un `FinancialDataRequest` estructurado, este contrato se valida a nivel formal y, despues, se comprueba operativamente mediante una descarga real en `yfinance`. Solo cuando ambas comprobaciones se superan se considera que el sistema dispone de una base suficientemente fiable para pasar al analisis.

La fase central del flujo comienza entonces con el Agente 2, que recibe los datos ya descargados y los convierte en un JSON analitico que acota que calculos deben realizarse. A partir de ese plan, el Agente 3 genera el script Python encargado de materializar el analisis. Ese script no se ejecuta directamente: primero pasa por el Agente 4, que valida su estructura y sus restricciones, y solo despues se lanza en el

ejecutor. Finalmente, el Agente 5 interpreta la salida correcta y la convierte en una respuesta legible para el usuario.

La arquitectura incorpora además cuatro puntos de corrección, uno por cada familia de fallos relevantes: errores estructurales en la petición de datos, errores operativos de descarga, errores estáticos del código generado y errores observados durante la ejecución. Esta separación de responsabilidades no solo ordena el sistema, sino que mejora su explicabilidad. Cada bloque tiene una entrada definida, una salida esperada y un criterio claro de éxito o bloqueo, lo que facilita defender por qué el flujo avanza, por qué se repara o por qué se detiene en un caso concreto.

5.4. Entrada y estado compartido

5.4.1. Entrada del sistema

El sistema arranca a partir de una estructura de entrada ligera, representada internamente como `FinancialQueryInput`. El único campo estrictamente obligatorio en ese punto es la consulta original del usuario, es decir, la petición en lenguaje natural sobre la que debe trabajar el flujo. Junto a ella pueden aparecer, si ya se conocen, algunos elementos de contexto como tickers sugeridos, un rango temporal, un periodo, un intervalo o rutas de CSV previamente disponibles. Sin embargo, estos campos auxiliares no sustituyen a la consulta principal ni se consideran conocimiento plenamente validado hasta que la fase de datos los revisa.

Esta decisión de diseño es importante porque evita que el sistema parta de supuestos opacos. Ni el Agente 1 ni los agentes posteriores deben asumir libremente que un ticker es correcto, que un periodo está bien resuelto o que existe ya un fichero útil si esa información no ha quedado consolidada dentro del flujo. La consulta del usuario es la fuente primaria de intención; el resto del contexto solo se convierte en contrato operativo después de la validación correspondiente.

También conviene subrayar que la entrada no incluye una etiqueta externa que indique de antemano la profundidad analítica esperada. El sistema no recibe una clasificación previa del tipo “consulta simple” o “consulta avanzada”. Esa graduación debe inferirse a partir del contenido de la petición, de sus restricciones y del formato de salida que se desprenda de ella. Este detalle es metodológicamente relevante porque muestra que el comportamiento de los agentes se apoya en el significado de la consulta y no en una señal manual añadida fuera del flujo.

Desde el punto de vista de la calidad, una entrada bien delimitada reduce varios riesgos tempranos: ambigüedades sobre el instrumento, incoherencias temporales, peticiones imposibles de descargar y errores derivados de trasladar al análisis hipótesis que todavía no han sido comprobadas. Por eso la memoria puede defender que el sistema

no empieza “analizando”, sino primero asegurando que la petición de partida tiene una forma utilizable y auditable.

5.4.2. Estado compartido

La ejecución se articula alrededor de un estado compartido tipado, `WorkflowState`, que actúa como memoria operativa del flujo. Su función no es solo transportar datos entre nodos, sino conservar artefactos y decisiones intermedias para que la respuesta final pueda reconstruirse después. Gracias a este estado común, cada fase trabaja sobre resultados ya consolidados de la fase anterior y deja, a su vez, una salida observable para las siguientes.

Un primer bloque del estado recoge la intención original y su normalización progresiva. Aquí se guardan la consulta del usuario y un contexto resuelto que va acumulando tickers, fechas, periodo, intervalo y rutas de CSV conforme la fase de datos los valida. Este grupo de campos permite distinguir entre lo que el usuario pidió inicialmente y lo que el sistema ha conseguido concretar de forma operativa.

Un segundo bloque conserva los artefactos de la fase de datos: el `FinancialDataRequest` validado, el resumen de descarga y las rutas de los ficheros persistidos. Esta información es la que hace posible enlazar la fase 1 con la fase 2 de manera explícita. El analista y el generador de código no reciben una consulta vaga, sino un contexto derivado de estos artefactos, resumido en un payload compacto pero respaldado por el estado completo.

El tercer bloque corresponde al núcleo de esta parte del flujo: `analysis_plan` y `generated_code`. El primero recoge el contrato analítico decidido por el Agente 2; el segundo almacena el script Python producido por el Agente 3. Ambos campos son esenciales para la trazabilidad interna, porque permiten separar que se quiso calcular de como se implementó técnicamente.

Por último, el estado incorpora la salida de ejecución, la respuesta final y los mecanismos de control. Aquí se incluyen `execution_stdout`, `execution_stderr`, `execution_output`, los artefactos de ejecución, la respuesta final en lenguaje natural, la lista de avisos acumulados, el mensaje de error si lo hubiera y el estado textual del workflow. Este conjunto permite reconstruir no solo los casos exitosos, sino también en qué punto se bloqueó el flujo y con qué motivo.

En consecuencia, el valor del estado compartido no reside únicamente en “pasar variables” entre nodos. Su papel principal es convertir una secuencia de llamadas a agentes y validadores en un proceso auditable, donde cada transición deja una evidencia concreta y donde la respuesta final puede justificarse a partir de artefactos observables.

5.5. Flujo principal del sistema

Este bloque debe contar el recorrido principal del sistema de forma limpia, sin detenerse todavia en todos los bucles de correccion. La idea es explicar que hace cada fase, que recibe, que produce y por que es necesaria dentro del flujo completo.

5.5.1. Parte de datos

La parte de datos cubre el tramo que va desde la consulta inicial del usuario hasta la obtencion de una descarga valida lista para el Agente 2. Su funcion no es producir todavia el analisis financiero, sino delimitar con precision sobre que informacion va a poder trabajar el sistema. Esta distincion es importante: antes de hablar de metricas, comparaciones o interpretaciones, el flujo debe demostrar que ha entendido que datos necesita, que puede pedirlos de forma coherente y que la fuente externa responde con una descarga util.

El primer paso de esta fase corresponde al Agente 1, que actua como planificador de datos. A partir de la consulta en lenguaje natural, este agente genera un `FinancialDataRequest` en formato JSON. Ese contrato debe recoger, al menos, los instrumentos o tickers detectados, la granularidad temporal, el rango o periodo solicitado y la indicacion de si la peticion esta lo bastante clara como para continuar. El objetivo aqui no es inferir conclusiones financieras, sino traducir la intencion del usuario a una peticion de datos concreta, utilizable por el resto del sistema y suficientemente explicita como para ser revisada.

Una vez generado ese request, el flujo no pasa todavia a la descarga. Primero se realiza una validacion estructural. En esta comprobacion se revisa que el JSON exista realmente, que sus campos obligatorios esten presentes, que los tipos sean correctos y que no haya incoherencias obvias entre fechas, periodos, intervalos o lista de instrumentos. Tambien se comprueba que la peticion no este apoyada en supuestos demasiado debiles. Si el request contiene errores corregibles, se activa el Subagente 1, cuya tarea consiste en reescribirlo de forma controlada y devolver una nueva version para validacion. Si el problema no es corregible con fiabilidad, o si se agotan los intentos permitidos, el flujo se bloquea y no avanza a etapas posteriores.

Superada la validacion estructural, la fase de datos entra en su segunda comprobacion: la validacion operativa de descarga. Aqui se intenta la descarga real en `yfinance` con los parametros ya validados. Esta decision es metodologicamente relevante, porque no basta con que la peticion sea correcta en teoria. Tambien debe demostrarse que produce un resultado util en la practica. En esta etapa se verifica que la llamada no devuelve un conjunto vacio, que los instrumentos esperados aparecen efectivamente y que la respuesta permite persistir artefactos de datos reutilizables por el sistema. Si esta descarga falla de forma reparable, interviene el Subagente 2, especializado en ajustar

la petición desde un punto de vista operativo; si no existe una corrección razonable, la ejecución vuelve a detenerse.

El resultado final de esta fase es, por tanto, más fuerte que un simple “request válido”. Lo que se entrega al Agente 2 es una descarga ya realizada, respaldada por un `FinancialDataRequest` previamente comprobado y acompañada de los artefactos necesarios para reconstruir el proceso. Desde el punto de vista de la explicabilidad, esta organización permite defender que el sistema no empieza a analizar ni a generar código sobre una interpretación vaga de la consulta, sino sobre datos cuya necesidad, forma de obtención y viabilidad han quedado justificadas antes de continuar.

5.5.2. Analista y generador de código

Una vez superada la fase de datos, el flujo entra en un segundo bloque conceptual: convertir una petición ya respaldada por datos descargados en un procedimiento analítico ejecutable. Esta transformación se divide en dos agentes consecutivos para separar con claridad dos responsabilidades distintas. El Agente 2 decide que debe calcularse; el Agente 3 decide como materializarlo en Python. Esta división no es ornamental, sino una medida de control. Evita pedir al mismo bloque que razone el análisis, improvise la implementación y produzca a la vez una conclusión final.

El Agente 2 actúa como analista. Su entrada no es la consulta bruta, sino un traspaso compacto construido a partir del estado de la fase 1. Ese payload contiene la consulta original, los tickers resueltos, el contexto temporal, las rutas de los CSV normalizados y un resumen mínimo de los datos reales disponibles, incluyendo número de filas y columnas observadas. Sobre esa base, el agente genera un `AnalysisPlan` estructurado. Este plan fija el objetivo analítico, el tipo de análisis, las métricas que deben calcularse, las columnas requeridas, las restricciones de datos, las exigencias de salida y una justificación breve de por qué ese contrato responde a la consulta. El resultado no es todavía una respuesta financiera, sino una especificación intermedia revisable.

Entre el Agente 2 y el Agente 3 existe una capa breve de validación local implementada en `src/analysis/validation.py`. Su función es comprobar que el plan no solo tiene forma de JSON correcta, sino que además es utilizable sobre la descarga real. En esta revisión se comprueba, entre otros aspectos, que el objetivo analítico no esté vacío, que `analysis_type` pertenezca a un catálogo permitido, que existan métricas y columnas requeridas y que dichas columnas aparezcan realmente en el conjunto normalizado descargado. También se exige que el plan describa restricciones de datos, requisitos de salida, preferencias de presentación y razonamiento mínimo. Esta validación cumple aquí el mismo papel metodológico que la validación estructural en la fase de datos: frenar el avance si el contrato generado por el LLM aun no es fiable.

Cuando esta comprobacion falla, el motivo no suele ser un error sintactico llamativo, sino una desalineacion entre el plan propuesto y el contexto real disponible. Puede ocurrir, por ejemplo, que el agente pida columnas que no existen en la descarga normalizada, que use un tipo de analisis fuera del catalogo permitido o que deje demasiado indeterminados los requisitos de salida. En ese caso, el flujo no intenta continuar hacia el codegen con un contrato dudoso. La transicion al Agente 3 se detiene, el estado registra el motivo del bloqueo y la ejecucion queda cerrada de forma controlada. Desde el punto de vista de la defensa del sistema, este comportamiento es importante porque muestra que la arquitectura prefiere interrumpirse antes que producir codigo sobre una base analitica no validada.

Solo cuando ese plan se considera valido interviene el Agente 3, encargado de generar el script Python. Su entrada combina tres piezas: la consulta original, el mismo payload compacto que vio el analista y el **AnalysisPlan** ya validado. El generador no debe reinterpretar la consulta desde cero ni decidir nuevas metricas, sino implementar el contrato recibido. Para ello se le impone un prompt con restricciones tecnicas explicitas: el script debe leerse desde `argv[1]`, cargar el payload mediante `Path(...).read_text(...)`, usar los cargadores permitidos del proyecto para acceder a los CSV, evitar accesos libres como `open()` o `pd.read_csv()` y producir un unico JSON por `stdout`. Ademias, el prompt delimita imports permitidos, claves esperadas en la salida y reglas de comportamiento cuando falten datos.

Esta relacion entre plan y script es uno de los puntos mas defendibles de la arquitectura. El **AnalysisPlan** actua como contrato entre razonamiento e implementacion: reduce la ambigüedad, facilita revisar si el codigo calcula lo que se pidio y permite aislar errores de planificacion de errores de programacion. La salida del Agente 3 se guarda primero en memoria compartida como **generated_code**; despues, cuando la parte de ejecucion toma el relevo, ese mismo contenido se persiste como fichero en `results/code/generated_<run_id>.py`. De esta manera, la fase no entrega una conclusion opaca, sino dos artefactos trazables: un plan analitico y un script verificable.

Los riesgos principales de esta traduccion aparecen precisamente en la frontera entre ambos agentes: planes demasiado vagos, columnas pedidas que no existen realmente, scripts que ignoran el contrato analitico o implementaciones que incumplen las reglas de carga y salida. Por eso esta parte del flujo no termina en la generacion del codigo, sino en la preparacion de un artefacto que todavia debe someterse a validacion estatica y, despues, a validacion de ejecucion. En terminos metodologicos, esto permite defender que el sistema no pide al LLM una recomendacion financiera directa, sino una secuencia controlada de contratos y evidencias intermedias.

5.5.3. Validacion del codigo

Completar aqui como se revisa el codigo antes de ejecutarlo. Interesa explicar que se comprueba, por que se comprueba y que tipo de errores o conductas se quieren bloquear: uso de operaciones no permitidas, incumplimiento del contrato de entrada o salida, accesos inseguros, estructura incorrecta o ausencia de controles minimos.

Este bloque es muy importante para defender el diseno del sistema. Desde el punto de vista de la memoria, aqui puedes explicar por que generar codigo no es suficiente y por que la validacion previa es una condicion necesaria para poder hablar de un flujo controlado.

5.5.4. Validacion de ejecucion

Completar aqui que ocurre una vez que el codigo ya se lanza: control del proceso, recogida de `stdout` y `stderr`, interpretacion del codigo de retorno, comprobacion de que la salida tenga el formato esperado y deteccion de fallos de calculo o inconsistencias.

Conviene dejar claro que ejecutar no equivale automaticamente a obtener un resultado valido. Esta fase debe mostrar que el sistema tambien controla lo que ocurre despues de la generacion de codigo y que la ejecucion se evalua como una evidencia tecnica, no solo como un paso ciego.

5.5.5. Interpretacion y respuesta final

Completar aqui como se transforma la salida estructurada en una respuesta final en espanol. Lo importante es explicar que esta etapa no recalcula ni inventa resultados, sino que interpreta lo ya obtenido y lo comunica de forma clara, prudente y ajustada a la consulta del usuario.

Este apartado es un buen lugar para destacar la explicabilidad externa del sistema: no solo interesa que el flujo sea auditable por dentro, sino que la respuesta final tambien sea comprensible para un lector humano y deje claros los resultados observados, su lectura y sus limitaciones.

5.6. Subagentes de correccion y recuperacion

Completar aqui el papel de los subagentes o mecanismos de reparacion. Conviene separarlos del flujo principal para que primero se entienda el recorrido base y despues se vea como reacciona el sistema cuando algo falla.

En este apartado deberias poder explicar cuando se activa cada correccion, que informacion recibe el subagente, que tipo de error intenta reparar y que criterio decide

si el flujo continua o termina en error. Tambien interesa comentar si las reparaciones afectan al plan, al codigo, a la ejecucion o solo a una parte concreta del proceso.

Metodologicamente, esta seccion es importante porque muestra que el sistema no solo tiene una ruta ideal, sino tambien una estrategia controlada para gestionar fallos.

5.7. Diseno de prompts y contratos de los agentes

Este es el lugar adecuado para incluir los prompts dentro de la memoria principal. No conviene tratarlos como un anexo aislado, sino como parte del diseno operativo del sistema.

Completar aqui que objetivo tiene cada prompt, que entrada recibe cada agente, que salida se le exige y que restricciones guian su comportamiento. Puedes apoyarte en una tabla resumen y despues desarrollar los agentes mas importantes de forma narrativa.

Agente	Entrada principal	Salida esperada	Aspectos del prompt que conviene explicar
Agente de analisis	Consulta original mas payload compacto con tickers, contexto temporal, CSV normalizado y columnas disponibles	AnalysisPlan en JSON tipado	Objetivo analitico, catalogo de tipos de analisis, columnas disponibles, metricas requeridas y contrato de salida estructurada.
Agente de codigo	Consulta original, payload compacto y AnalysisPlan validado	JSON con un unico campo code que contiene el script completo	Contrato tecnico de lectura del payload, cargadores permitidos, imports autorizados, salida JSON unica y restricciones de implementacion.
Agente de interpretacion	<i>Completar</i>	<i>Completar</i>	<i>Uso exclusivo de resultados ejecutados, tono, prudencia y estructura de respuesta.</i>
Subagentes de correccion	<i>Completar</i>	<i>Completar</i>	<i>Tipo de error que reciben, alcance de la reparacion y limites de actuacion.</i>

Tabla 5.1: Guia para documentar los prompts y contratos de los agentes.

En este bloque debes poder defender no solo que existen prompts distintos, sino por que estan separados y que gana el sistema con esa especializacion. Si incluyes fragmentos textuales, conviene que sean los que mejor revelan el contrato de cada agente y no necesariamente el prompt completo sin explicacion.

5.8. Trazabilidad y artefactos generados

Uno de los rasgos mas importantes del sistema es que cada ejecucion deja una cadena de artefactos que permite reconstruir el flujo completo. Esta trazabilidad no depende de logs narrativos genericos, sino de objetos y ficheros con una funcion concreta dentro de la arquitectura. Por eso puede afirmarse que la respuesta final no aparece como un resultado opaco, sino como la ultima pieza de una secuencia de evidencias observables.

En la transicion entre la fase de datos y la parte analitica, la primera evidencia relevante es el contexto resuelto que queda en `WorkflowState`. A partir de ese estado se construye el payload compacto que conecta la fase 1 con los agentes 2 y 3. Aunque ese payload no persiste inicialmente como fichero independiente, su contenido puede reconstruirse porque procede de campos tipados del estado, del resumen de descarga y de las rutas de los CSV normalizados.

El siguiente artefacto clave es el `AnalysisPlan`. Este objeto no se trata como una simple respuesta textual del LLM, sino como un contrato analitico formal que se parsea, se tipa y se valida antes de permitir el codegen. Gracias a ello, el sistema conserva una evidencia explicita de que calculos se querian realizar, con que columnas y con que formato de salida esperado. Desde el punto de vista metodologico, este punto es central: permite discutir por separado si el plan era adecuado y si el codigo lo implemento correctamente.

La segunda evidencia principal es el script generado por el Agente 3. Durante la parte 2 este codigo vive en memoria compartida como `generated_code`. Cuando la parte de ejecucion toma el relevo, el sistema lo persiste en `results/code/generated_<run_id>.py`. Junto a el se guarda tambien el payload efectivo de ejecucion en `results/logs/payload_<run_id>.json`, lo que permite revisar exactamente con que contexto operativo se lanzo el script.

Tras la ejecucion aparecen otros artefactos igualmente relevantes: `stdout`, `stderr`, el codigo de retorno y la salida parseada del script. Estos elementos se persisten en rutas como `results/logs/stdout_<run_id>.json` y `results/logs/stderr_<run_id>.log`, y se reflejan ademas en el estado compartido. Gracias a ello puede saberse no solo si el script produjo un resultado, sino tambien si fallo, por que fallo y si la salida respetaba o no el contrato esperado.

Finalmente, la lista de avisos, el mensaje de error y la respuesta final completan la trazabilidad funcional del sistema. Los avisos permiten registrar irregularidades no bloqueantes; el mensaje de error localiza la causa principal cuando el flujo se interrumpe; y la respuesta final muestra como se ha traducido la salida estructurada a lenguaje natural. En conjunto, estos artefactos permiten defender los resultados del TFM con evidencias inspeccionables y no solo con una impresion subjetiva de que “el sistema parece responder bien”.

5.9. Decisiones de diseno

Completar aqui por que se elige este flujo y no otro. Este apartado debe servir para justificar las principales elecciones de arquitectura: separacion de responsabilidades, uso de varios agentes, validacion previa a la ejecucion, reparacion controlada, anclaje a datos locales y conservacion de artefactos.

No se trata de repetir la metodologia, sino de defender las decisiones tecnicas que hacen posible luego una evaluacion seria del sistema. En otras palabras, aqui explicas por que la arquitectura esta pensada para producir un analisis mas controlable, mas explicable y mas revisable que una respuesta directa generada de una sola vez por el modelo.

Capítulo 6

Resultados y evaluacion

6.1. Funcion de este capitulo

Este capitulo debe mostrar que ha ocurrido al ejecutar el sistema y como deben interpretarse esos resultados. Su objetivo no es solo presentar metricas, sino explicar que ensena la evaluacion sobre la robustez del flujo y sobre el valor real del TFM.

6.2. Enfoque recomendado

De momento conviene dejar este capitulo planteado, no cerrado. Cuando llegue el momento, la redaccion final deberia combinar:

1. una lectura global del rendimiento;
2. un analisis por tipo de caso o por grado de exigencia de la consulta;
3. una lectura por etapas del flujo;
4. una discusion de fallos, reparaciones y limites.

6.3. Preguntas que convendra responder mas adelante

1. ¿Que significa exactamente que el sistema haya completado bien un caso?
2. ¿Donde se concentran los errores: datos, planificacion, generacion, validacion, ejecucion o interpretacion?
3. ¿Que gana el sistema con los subagentes de correccion?
4. ¿Que tipos de consulta tensan mas la arquitectura?

5. ¿Que parte de los resultados respalda de verdad la tesis del trabajo?

6.4. Plantilla de estructura futura

6.4.1. Resultado global

Completar aqui el resumen global de la evaluacion cuando el sistema este implementado y probado.

6.4.2. Lectura por grupos de casos

Completar aqui la segmentacion de consultas que resulte mas util: por tipologia, por numero de restricciones, por formato de salida exigido o por etapa tensionada del flujo.

6.4.3. Analisis por etapas

Completar aqui el rendimiento observado en cada bloque del flujo.

6.4.4. Casos representativos

Completar aqui varios ejemplos que ilustren un caso sencillo, uno intermedio, uno complejo resuelto y un fallo informativo.

6.4.5. Discusion critica

Completar aqui que se ha demostrado, que no se ha demostrado y que mejoras quedarian priorizadas para una siguiente iteracion.

Capítulo 7

Limitaciones, aspectos eticos y reproducibilidad

7.1. Funcion de este capitulo

Este capitulo reúne tres elementos que no son el centro del TFM, pero si condicionan su interpretacion final: las limitaciones del estudio, el uso responsable del sistema y las condiciones para repetir el trabajo.

7.2. Limitaciones del estudio

Completar aqui limites de datos, dependencia del modelo, cobertura de casos, sensibilidad a prompts, dificultad de generalizacion y cualquier restriccion importante del sistema. Este es el lugar principal para reunir las limitaciones globales del TFM, aunque en metodologia y resultados tambien puede ser util comentar limites especificos de cada bloque.

7.3. Aspectos eticos y uso responsable

Completar aqui finalidad prevista, usos excluidos, supervision humana, limites del analisis historico y riesgos de ejecutar codigo generado o de sobrerinterpretar resultados financieros.

7.4. Reproducibilidad

Completar aqui lo necesario para repetir el trabajo: entorno, dependencias, datos, artefactos conservados, variables de entorno, pasos minimos de ejecucion y factores que pueden hacer variar el resultado.

7.5. Preguntas de control

1. ¿Que no debería concluir un lector a partir del sistema?
2. ¿Que necesita exactamente otra persona para repetir un caso?
3. ¿Que parte del resultado depende del flujo y que parte depende del modelo concreto?
4. ¿Que riesgos siguen abiertos aunque el sistema funcione?

Capítulo 8

Conclusiones y trabajo futuro

8.1. Funcion de este capitulo

La conclusion final debe recoger la aportacion principal del TFM con una formulacion clara y defendible. No conviene que sea una repeticion de capitulos anteriores ni una lista dispersa de ideas futuras.

8.2. Guia para las conclusiones

Completar aqui, al final del proyecto, una sintesis que responda a estas preguntas:

1. ¿Que problema has abordado realmente?
2. ¿Que has conseguido demostrar?
3. ¿Que evidencia es la mas importante para sostener esa afirmacion?
4. ¿Que limite obliga a interpretar el resultado con prudencia?

8.3. Guia para el trabajo futuro

Completar aqui las siguientes lineas de trabajo priorizadas: mejoras del flujo, robustez, validacion, ampliacion de casos, comparativas con otras soluciones y posible evolucion del sistema.

Apéndice A

Anexos

A.1. Ejecucion del sistema

Este anexo conserva solo la informacion operativa que sigue siendo util para ejecutar y defender la version actual del proyecto. El punto de entrada principal es `run_mvp.py`, que recibe una consulta financiera estructurada, construye el workflow multiagente y devuelve un JSON final con el estado completo de la ejecucion.

A.1.1. Preparacion minima

Antes de lanzar una ejecucion conviene comprobar cuatro condiciones:

1. que el entorno Python tenga instaladas las dependencias del proyecto;
2. que el fichero `.env` contenga una configuracion LLM valida;
3. que los CSV historicos existan en `data/raw`;
4. que la consulta de entrada incluya las rutas reales a esos CSV.

Una instalacion minima puede hacerse con:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

A.1.2. Ejecucion con ejemplos integrados

El proyecto incluye varios casos ya preparados en `src/examples/sample_inputs.py`. Cada uno contiene la consulta, los tickers y las rutas locales a los datos necesarios. La forma mas simple de comprobar el sistema es ejecutar uno de esos ejemplos:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example complex_nvda_growth_report
```

Esta via es la mas util para una primera validacion, porque evita errores de formato en la entrada y permite comprobar el flujo de extremo a extremo con casos ya definidos dentro del repositorio.

A.1.3. Ejecucion con una entrada JSON propia

Tambien es posible lanzar el sistema con un fichero JSON preparado manualmente:

```
python run_mvp.py --input-json ruta\al\caso.json
```

La estructura minima esperada es la siguiente:

```
{
  "query": "Analiza los retornos de QQQ y SPY en 2024",
  "tickers": ["QQQ", "SPY"],
  "csv_paths": [
    "C:/ruta/al/repositorio/data/raw/qqq_y_spy_desde_20240101_hasta_20241231.csv"
  ],
  "start": "2024-01-01",
  "end": "2024-12-31",
  "period": null,
  "interval": "1d",
  "needs_clarification": false,
  "warnings": []
}
```

Los campos `query`, `tickers` y `csv_paths` son los que definen el caso de forma mas directa. El resto de atributos ayudan a restringir el periodo, la granularidad temporal y las advertencias que se quieran conservar desde la entrada.

A.1.4. Salida y artefactos generados

La ejecucion imprime por consola un JSON con el estado final del workflow. Ese estado incluye, entre otros elementos:

- la consulta original y su forma normalizada;
- el plan analitico generado por el agente planificador;
- el codigo Python producido para el calculo;
- la salida estructurada de la ejecucion;

- la respuesta final redactada para el usuario;
- el estado de la ejecucion, los avisos y los posibles errores.

Ademas, el sistema conserva artefactos intermedios en dos carpetas:

- `results/code`, con el codigo generado en cada ejecucion;
- `results/logs`, con el payload enviado al ejecutor y los ficheros de `stdout` y `stderr`.

Esta conservacion resulta util tanto para depurar fallos como para defender metodologicamente la trazabilidad del flujo en la memoria.

A.2. Configuracion LLM y API

La aplicacion utiliza un endpoint compatible con OpenAI Chat Completions. En la configuracion actual, el modelo `openai/gpt-oss-20b` se sirve a traves de `vLLM`, aunque el codigo acepta varias familias de variables de entorno equivalentes.

A.2.1. Configuracion recomendada

La forma mas directa de configurar el sistema es declarar en `.env`:

```
LLM_PROVIDER=openai-compatible
VLLM_BASE_URL=https://servidor-vllm.example/v1
VLLM_API_KEY=pegar_aqui_la_clave
VLLM_MODEL=openai/gpt-oss-20b
```

A.2.2. Variables equivalentes admitidas

El codigo reconoce tres familias de nombres para base URL, clave y modelo:

- familia `VLLM_*`: `VLLM_BASE_URL`, `VLLM_API_KEY`, `VLLM_MODEL`;
- familia `LLM_*`: `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`;
- familia `OPENAI_*`: `OPENAI_BASE_URL`, `OPENAI_API_KEY`, `OPENAI_MODEL`.

La resolucion de configuracion prioriza primero los valores `VLLM_*`, despues los valores genericos `LLM_*` y finalmente los alias `OPENAI_*`.

A.2.3. Parametros de generacion

Tambien pueden ajustarse los parametros principales del cliente:

```
LLM_TEMPERATURE=0.1  
LLM_MAX_TOKENS=4096  
LLM_TIMEOUT_SECONDS=120  
OPENAI_VERIFY_SSL=true
```

Estos valores controlan la temperatura de generacion, el maximo de tokens de salida, el tiempo de espera del cliente y la verificacion SSL del endpoint remoto.

A.2.4. Comprobaciones practicas

Si la configuracion es correcta, una ejecucion con `run_mvp.py` debe completar el flujo sin necesidad de descargar datos externos. Si la clave no esta definida o contiene un valor de ejemplo, el sistema no intenta llamar al modelo y devuelve un error de configuracion explicito.

A.3. Notas para reproducibilidad

De cara a la redaccion final de la memoria, este anexo sirve como referencia corta de reproduccion operativa. Para repetir un caso hacen falta el mismo codigo, los mismos CSV, una configuracion equivalente del modelo y acceso a un endpoint compatible con OpenAI. Si alguno de estos elementos cambia, el comportamiento del sistema puede variar incluso aunque la arquitectura general siga siendo la misma.